

ChatGPT - IUTCDI

Lecture 03 EDD.pdf

PDF

read and understand this. dont say anything

Lecture 03 EDD

explain what is EDD

EDD (Evolutionary Database Design) is a **database design and maintenance approach** where the database schema evolves gradually through small, controlled changes as application requirements change over time. Instead of designing the entire database perfectly at the beginning, EDD accepts that software systems continuously evolve, so the database must evolve safely with them.

Lecture 03 EDD

The lecture clearly mentions that EDD:

- **is not a database design pattern**
- **is a process for maintaining databases in evolving systems**
- focuses on continuous updates and adaptation to customer requirements.

Lecture 03 EDD

Main idea of EDD

Traditional databases were often changed manually and separately from application development, which caused conflicts, inconsistencies, and deployment issues.

EDD solves this by treating database changes just like software code changes.

Core practices of EDD

1. Database changes are written as migrations

Every schema change is stored in migration scripts and version controlled together with application code.

Lecture 03 EDD

A migration:

- contains SQL commands,
- has a unique ID,
- is stored in a migrations folder,
- and is tracked using a **ChangeLog** table.

Lecture 03 EDD

Example:

SQL

```
ALTER TABLE users ADD COLUMN preferred_language VARCHAR(10);
```

2. Developers and DBAs collaborate

Developers know what functionality is needed, while DBAs understand the overall database structure and performance concerns. EDD encourages close cooperation between them.

Lecture 03 EDD

3. Everyone has their own database instance

Each developer works on a private database copy to avoid conflicts and allow safe experimentation.

Lecture 03 EDD

4. Continuous integration of database changes

Database updates are integrated frequently in small increments to avoid large, difficult merges later.

Lecture 03 EDD

5. Database refactoring

EDD treats schema changes as **refactorings** involving:

1. schema changes,
2. data migration,
3. application/database access code updates.

Lecture 03 EDD

So changing a database is not only modifying tables — the related code and existing data must also be updated safely.

Types of changes in EDD

The lecture divides database changes into two categories:

Lecture 03 EDD

A. Non-destructive changes

These do not break existing systems.

Examples:

- adding a new table,
- adding a nullable column,
- adding columns with default values.

Old code continues to work normally.

B. Destructive changes

These can break old applications if done directly.

Example:

Renaming a column from **phone** to **phone_number**.

Lecture 03 EDD

EDD handles this using a **3-phase migration**:

1. Add the new column.
2. Keep both old and new columns temporarily (dual-write period).
3. Remove the old column only after all code is updated.

This avoids downtime and compatibility problems.

Tools commonly used in EDD

The lecture mentions:

- Liquibase
- Active Record Migrations

Lecture 03 EDD

These tools automate migration and refactoring processes.

In simple words

EDD means:

“Change the database slowly, safely, and continuously as the software evolves.”

Instead of making huge risky database redesigns, EDD encourages:

- small incremental updates,
- version-controlled migrations,
- teamwork,
- automation,
- and safe refactoring practices.

so explain the whole slide please

Evolutionary Database Design (EDD) — Full Slide Explanation

1. What is Evolutionary Database Design?

EDD is a way of managing databases where the database continuously changes and improves as software requirements evolve.

Lecture 03 EDD

Traditional database design assumed:

- requirements are fixed,
- database structure rarely changes.

But in real software projects:

- customer needs change,
- new features are added,
- old features are modified,
- applications grow continuously.

So EDD accepts that:

“Database evolution is normal.”

Instead of redesigning the whole database repeatedly, EDD promotes:

- small,
- safe,
- incremental changes.

2. What EDD is About

The slide says:

- EDD is **not** a database design pattern.
- It is a **design process** for maintaining databases in evolving systems.

Lecture 03 EDD

Meaning:

- EDD is not a specific schema structure.
- It is a methodology or workflow for handling database changes over time.

The focus is:

- adaptability,
- maintainability,
- continuous improvement.

3. DBA and Developer Collaboration

The lecture says DBAs and developers must work closely together.

Lecture 03 EDD

Why?

Developers know:

- what new feature is needed,
- what the application requires.

DBAs know:

- overall database structure,
- optimization,
- indexing,
- performance,
- integrity,
- relationships with other systems.

Together they can:

- make safer changes,
- avoid performance issues,
- maintain consistency.

Without collaboration:

- developers may create inefficient schemas,
- DBAs may misunderstand application requirements.

4. Version Control for Database Artifacts

EDD says:

All database artifacts must be version controlled with application code.

Lecture 03 EDD

Database artifacts include:

- schema scripts,
- migrations,
- triggers,
- views,
- stored procedures,
- seed data.

Everything should be stored in:

- Git,
- GitHub,

- GitLab,
 - Bitbucket,
 - etc.
-

Why is this important?

Because:

- there is one source of truth,
 - changes are traceable,
 - rollback becomes easier,
 - deployment becomes safer,
 - developers stay synchronized.
-

5. Database Changes as Migrations

This is one of the most important EDD concepts.

Previously:

- developers changed databases manually,
- DBAs later compared development DB with production DB,
- changes were manually recreated.

Problems:

- error-prone,
- confusing,
- hard to track,
- risky.

Lecture 03 EDD

EDD Solution: Migration Scripts

Every change is written as a migration script.

Lecture 03 EDD

Example:

SQL

```
ALTER TABLE users ADD COLUMN age INT;
```

Each migration:

- has a unique ID,

- stored in migration folder,
- version controlled,
- applied sequentially.

6. ChangeLog Table

EDD tracks applied migrations using a **ChangeLog** table.

Lecture 03 EDD

The screenshot on page 6 shows:

- migration number,
- application date,
- applied by,
- description.

Purpose:

- know which migrations ran,
- prevent duplicate execution,
- maintain deployment order.

7. Everyone Gets Their Own Database Instance

The lecture emphasizes:

Every developer should have a private database instance.

Lecture 03 EDD

Why?

If everyone shares one development database:

- unfinished work conflicts,
- broken schema affects others,
- testing becomes unreliable.

Private databases allow:

- isolated experimentation,
- safe testing,
- uninterrupted development.

The diagram on page 7 shows each developer having their own schema/database.

8. Continuous Integration of Database Changes

Developers frequently integrate database changes into a shared integration database.

Lecture 03 EDD

The diagram on page 8 shows:

- developer creates migration,
- commits to Git,
- CI server applies migrations,
- integration database updates automatically.

Why Continuous Integration?

Because small frequent updates:

- reduce merge conflicts,
- simplify debugging,
- catch errors early,
- reduce deployment risk.

This follows modern DevOps practices.

9. Database = Schema + Data

EDD says:

A database consists of both schema and data.

Lecture 03 EDD

Not only table structures matter.

Also important:

- static data,
- lookup tables,
- testing data,
- legacy migrated data.

Examples:

- country list,
- product categories,
- user roles.

These must also evolve safely.

10. Database Refactoring

EDD treats database changes as refactorings.

Lecture 03 EDD

A refactoring includes 3 things:

1. changing schema,
 2. migrating existing data,
 3. updating application access code.
-

Example

Suppose:

SQL
phone

becomes:

SQL
phone_number

You must:

- add new column,
 - move old data,
 - update application queries,
 - update APIs,
 - then safely remove old column.
-

11. Automating Refactorings

The lecture mentions tools like:

- Liquibase,
- Active Record Migrations.

Lecture 03 EDD

These tools:

- automate migrations,
- track versions,
- simplify deployment.

Benefits:

- fewer manual errors,

- consistent deployments,
 - easier rollback.
-

12. Separate Database Access Layer

EDD recommends:

clearly separating DB access code.

Lecture 03 EDD

Meaning:

- application should not directly manipulate database everywhere.

Instead:

- use repositories,
- DAO layer,
- ORM,
- service layer.

Benefits:

- easier maintenance,
 - easier schema changes,
 - less SQL knowledge required for developers.
-

13. Release Frequently

EDD encourages frequent releases.

Lecture 03 EDD

Reason:

- small releases are safer,
- easier to test,
- easier rollback,
- quicker feedback.

Large database releases are dangerous because:

- failures become catastrophic,
 - debugging becomes harder.
-

14. Types of Database Changes

The lecture divides changes into:

A. Non-Destructive Changes

Safe changes that do not break old code.

Examples:

- adding nullable column,
- adding new table.

Example from slide:

SQL

```
ALTER TABLE users  
ADD COLUMN preferred_language VARCHAR(10) NULL;
```

Why safe?

- old code ignores new column,
 - existing data unaffected.
-

B. Destructive Changes

Changes that can break existing systems.

Examples:

- dropping columns,
- renaming columns,
- changing data type.

These require a special process.

15. 3-Phase Migration for Destructive Changes

Example:

Renaming:

SQL

```
phone → phone_number
```

EDD uses 3 phases.

Phase 1 — Start

Add new column:

SQL

```
ALTER TABLE users  
ADD COLUMN phone_number VARCHAR(20);
```

Copy old data:

```
SQL  
UPDATE users  
SET phone_number = phone;
```

New code writes to both columns.

Phase 2 — Transition

Application temporarily supports both:

```
SQL  
SELECT COALESCE(phone_number, phone)
```

This ensures compatibility.

Phase 3 — End

After all systems use new column:

```
SQL  
ALTER TABLE users DROP COLUMN phone;
```

Safe removal.

16. Example: Splitting One Column into Multiple Columns

The slide shows:

```
SQL  
inventory_code
```

containing:

```
CTG_023_768
```

EDD refactors it into:

- location_code,
- batch_number,
- serial_number.

Steps:

1. add new columns,
2. extract data,
3. migrate indexes,

4. drop old column.

Lecture 03 EDD

This improves:

- searching,
- indexing,
- normalization.

17. Example: Many-to-Many Relationship

Previously:

SQL

```
products.category
```

Only one category per product.

EDD converts it into:

- **categories** table,
- **product_categories** join table.

Lecture 03 EDD

Benefits:

- one product can have multiple categories,
- normalized structure,
- scalable design.

18. Adding NOT NULL Columns Safely

Dangerous approach:

SQL

```
ALTER TABLE orders
ADD COLUMN status VARCHAR(20) NOT NULL;
```

Old code may fail because it doesn't provide value.

Safe EDD approach:

SQL

```
ALTER TABLE orders
ADD COLUMN status VARCHAR(20)
NOT NULL DEFAULT 'pending';
```

Old code still works because default value is inserted automatically.

Lecture 03 EDD

Later:

SQL

```
ALTER TABLE orders
ALTER COLUMN status DROP DEFAULT;
```

19. Final Steps in Refactoring

Page 16 explains remaining tasks:

- update stored procedures/views/triggers,
- update indexes,
- run tests,
- commit all changes together.

Lecture 03 EDD

This ensures:

- consistency,
 - reliability,
 - safe deployment.
-

Overall Philosophy of EDD

EDD promotes:

- continuous evolution,
- small incremental changes,
- automation,
- collaboration,
- safe refactoring,
- version control,
- continuous integration.

Main goal:

Make database evolution safe, manageable, and compatible with modern agile software development.